QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control **3**
PRELIMINARY ROUND (Test Strategy & Planning) **3**
1. Scope & Requirements Traceability 3
2. Risk Assessment & Mitigation Strategy 3
3. Test Environment & Execution Strategy 5
4. CI/CD Release Thresholds & Automation Gates 5
5. Test Case Specifications 6
6. Test Strategy & Plan Sign-Off 7

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	HalalChain Agent – AI-Powered Halal Compliance Workflow Orchestrator
Team Repo URL	HalalChain Team Repo
Project Board URL	HalalChain Agent Project Board

Objective:

The primary objective is to ensure that HalalChain Agent reliably orchestrates document classification, ingredient verification, supplier resolution, and JAKIM export generation with Google Gemini as the central reasoning engine. The system must handle ambiguous inputs, missing data, and workflow failures while maintaining audit transparency, enforcing compliance gates, and preventing non-Halal submissions.

PRELIMINARY ROUND (Test Strategy & Planning)
1. Scope & Requirements Traceability

This section aligns the testing connected back to specific user requirements via a "Requirement Traceability Matrix". So, it ensures every test has been conducted to prevent bugs/failure in products for that specific requirements and unplanned feature creep.

1.1 In-Scope Core Features

- Smart Document Upload & Classification: Drag-and-drop ingestion, OCR/text extraction, Gemini-powered categorization into JAKIM-required document types.
- Compliance Gap Checker: Real-time checklist validation against mandatory JAKIM submission requirements with missing item alerts.
- Ingredient Halal Verification: Gemini extraction & classification of ingredients into Halal/ Ambiguous/ Haram with confidence scoring and manual override modal.
- Supplier Resolution Loop: Structured request routing, supplier declaration form, mock cross-user sync, and audit logging.
- Structured JAKIM Export Generator: Schema-compliant JSON/PDF bundle generation with full Gemini reasoning audit trail.
- Model Centrality Proof: Workflow orchestration, confidence-based routing, and fallback logic entirely dependent on Google Gemini 1.5 Pro (approved alternative).

1.2 Out-of-Scope

- Real JAKIM MYeHALAL API integration (mock adapter used for hackathon)
- Multi-language support beyond English/Bahasa Malaysia (MVP scope)
- Production-grade authentication (SSO, 2FA) – email-based authentication only
- Mobile native app – responsive web only
- Payment/subscription features
- Advanced analytics dashboard – basic progress view only
- Multi-user collaboration workflows – single-user focus for MVP

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Risk Assessment & Mitigation Strategy

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
Gemini hallucinates Halal status (e.g., marks haram ingredient as halal)	3	5	15 (High)	• Output validation schema + mock JAKIM cross-check • Confidence threshold <60% → auto-flag for manual review • Clear disclaimer: "Final authority remains with JAKIM"	• Adversarial prompt testing (AI-03) • Hallucination detection unit tests • Manual override flow validation
Ambiguous ingredient not resolved (e.g., "minyak sapi" remains ambiguous)	4	4	16 (Critical)	• Manual Halal Checker modal with confirmation • Supplier resolution loop with deadline tracking • Export blocked until all ambiguous items resolved	• Edge case test: unresolved ambiguous item blocks export (TC-06) • Supplier sync simulation test (TC-08)
Missing required document not detected	3	4	12 (High)	• Gemini-powered gap analysis against JAKIM checklist • Visual alert banner + disabled export button • Checklist auto-update on upload	• Negative test: upload 2/3 required docs → alert shown (TC-04) • Regression test on checklist logic
Gemini timeout or API failure	2	5	10 (Medium)	• Cached fallback rules for common ingredients • Queue sync for later + user messaging: "Using cached data" • Retry logic with exponential backoff (mock)	• Simulate API timeout → verify fallback behavior (TC-10) • Load test with delayed Gemini responses
OCR fails on poor-quality invoice	3	3	9 (Medium)	• Gemini applies fuzzy matching + confidence scoring	• Test with blurry/low-res images (TC-02)

© UMHackathon 2026

Email: umhackathon@um.edu.my

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
image				• Prompt user: "Could not read clearly. Try re-uploading?" • Allow manual text entry fallback	• Validate manual entry fallback path
Export bundle violates JAKIM schema	2	4	8 (Medium)	• JSON schema validation layer before export • Gemini prompt enforces strict output format • Sample export validated against mock JAKIM spec	• Schema validation unit tests • Export test: verify JSON structure matches spec (TC-09)
User declares incorrect Halal status manually	3	3	9 (Medium)	• GLM validates manual declaration against known rules • Warning if declaration inconsistent with notes • Audit trail logs all manual overrides	• Test manual declaration with invalid notes (AI-03) • Verify audit log captures override

Risk Assessment Scoring Criteria:

Likelihood (1-5)		Severity (1-5)	
1	Rare	1	Impact is Negligible
2	Unlikely	2	Impact is Minor
3	Possible	3	Moderate Impact
4	Likely	4	Major Impact
5	Almost Certain	5	Critical Failure of the system

Risk Level Reference:

Risk Score	Risk Level	Recommended Action
1 – 5	Low	Monitor only. Acceptable risks.
6 – 10	Medium	Mitigate + Testing
11 – 15	High	Must need mitigating and through testing is required
16 – 25	Critical	Priority is Highest. Need extensive level of testing.

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Test Environment & Execution Strategy

- **Unit Test**
  - Scope: Document classification logic (detectType()), ingredient verification mock (glmSimulate('verify')), manual checker validation, export schema validation
  - Execution: Tests are written using Jest framework. Execution happens while developing as well as CI pipeline upon push to repo via GitHub Actions
  - Isolation: To focus only on the logic, external dependencies (e.g., FileReader API, mock JAKIM DB) are mocked using Jest mock functions
  - Pass Condition: Achieved happy cases, negative cases with proper return format (e.g., "Ingredient status must be halal/haram/ambiguous") and considered edge cases (empty input, malformed JSON).

- **Integration Test**
  - Scope: Upload → Classify → Checklist Update flow; Ingredient Verification → Manual Override → Export Eligibility flow; Supplier Request → Response → SME Sync flow
  - Execution: This test can be performed after proper completion of Unit Testing like "Merging SME Dashboard and Supplier Portal branches"
  - Workflow: To ensure the API calls and interactions with localStorage are working, real calls are tested without mocking for state persistence
  - Pass Condition: When integrated components like "document upload to ingredient verification" are working together correctly and showing expected outcome (e.g., haram item blocks export).

- **Test Environment (CI/CD Practice):**
  - Local Testing: This testing is manual one on localhost using Chrome DevTools and localStorage inspector
  - Staging/CI: GitHub Actions triggers tests on every push of code to develop branch
  - CI/CD automated pipeline: So, GitHub Actions does basic linting tests for branch specific levels and then Jest/Cypress for PR (Pull requests) to main branch.

- **Regression Testing & Pass/Fail Rules:**
  - Execution Phase: This testing tests across all 3 in-scope features each time a new branch is merged to main branch
  - Pass/Fail Condition: Test case pass is only considered when actual outcome perfectly aligns with expected one. If mismatch/threshold not achieved, the test status is considered fail and need to incorporate that in Defect Log
  - Continuation Rule: While doing testing in a gradual manner, the initial cases must need to meet the threshold to allow continue for further testing (e.g., GLM centrality test must pass before testing export flow)

© UMHackathon 2026

Email: umhackathon@um.edu.my

- **Test Data Strategy:**
    - Manual: Created 2 Mock Users account with different roles: SME Food Producer (keropok-mak-cik / Reg: 20240101 (1234567-X)) and Ingredient Supplier (xyz-ingredients / Reg: 20230109 (9876543-M))
    - Automated: JavaScript seed scripts (seed-test-data.js) can be used as input for the automated uses of dummy data (mock documents, ingredient lists, supplier declarations)
    - Mock JAKIM Database: JSON fixture with 50 common Malaysian food ingredients with statuses (halal/haram/ambiguous) for consistent testing

- **Passing Rate Threshold:**
    - A minimum of 86.2% of all test cases that are executed must pass. For the critical tests, all the tests (100%) need to be passed and there is no exception for that. Because critical failure costs huge in production (non-compliant Halal submission)

4. CI/CD Release Thresholds & Automation Gates

4.1 Integration Thresholds (Merging to Main)

Checks	Requirements	Project	Pass/Failed
Automatic Build	Zero Build Error	0 Errors	Passed
Unit Tests	100% Passing Rate	100%	Passed
Code Quality	Zero Linting Error	98% compliance	Passed
Test Coverage	Minimum 81.2%	84.3%	Passed
GLM Prompt Validation	All prompts return valid JSON schema	100%	Passed
Accessibility Check	WCAG 2.1 AA compliance for core flows	95%	Passed

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.2 Deployment Thresholds (Pushing to Production)

Checks	Requirements	Project	Pass/Failed
Regression Test	Minimum 90%	93.1%	Passed
AI Output Pass Rate	Minimum of 80% of documented prompt	87.5%	Passed
Critical Bugs	Zero P0/01 Bugs	0	Passed
API Performance	Response Time < 800ms	780ms avg	Passed
Security	API keys are not exposed, localStorage sanitized	Clean	Passed

5. Test Case Specifications

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	Happy Case (Entire Flow): SME Document to Export	Verify SME can upload docs, Gemini classifies, ingredients verified, export generated	1. Login as SME 2. Upload 3 sample docs 3. Click "Process with Gemini" 4. Navigate to Ingredient Verification 5. Resolve ambiguous items via Manual Check 6. Click "Export JSON"	Documents classified correctly • Ingredients show status/confidence • Export button enabled after resolution • JSON file downloaded with valid schema	All steps passed. Export JSON validated against mock JAKIM schema. Status: Passed
TC-02	Specific Case (Negative): Poor-Quality OCR Input	Verify system handles blurry/unreadable invoice gracefully without crashing	1. Upload low-res/blurred image file 2. Trigger Gemini classification	• System shows "Could not read clearly" warning • Offers "Retry Upload" or	System showed warning + manual entry option. App remained

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
				"Manual Entry" • No 500 error or crasH	stable. Status: Passed
TC-03	Specific Case (Negative): Missing Required Document	Verify gap checker blocks export if required documents are missing	1. Upload only 1/3 required docs 2. Click "Process" 3. Attempt to export	• Alert: "Missing: Premises License, Ingredient List" • Export button disabled • Checklist shows 1/3 complete	Alert shown, export disabled. Checklist updated accurately. Status: Passed
TC-04	Specific Case (Edge): Haram Ingredient Detected	Verify haram item blocks export + suggests compliant alternative	1. Upload ingredient list containing "Gelatin (porcine)" 2. Process with Gemini	• Ingredient card shows Haram + red border • Alert banner: "Non-Compliant Ingredient Detected" • Export button disabled • Suggests "agar-agar" alternative	All conditions met. Alternative suggestion displayed. Status: Passed
TC-05	NFR (Performance): Gemini Simulate Latency	Verify mock Gemini responses complete within threshold under load	1. Trigger ingredient verification for 10 items 2. Measure total processing time 3. Monitor UI responsiveness	• Total time < 10s • Progress bar updates smoothly • No UI freeze	Avg time: 7.2s. UI remained responsive throughout. Status: Passed
TC-06	NFR (Load): Concurrent File Uploads	Verify system handles multiple file uploads without degradation	1. Upload 5 files simultaneously via drag-and-drop 2. Monitor file list and checklist updates	• All files processed and classified • No duplicate entries • Checklist updates correctly	All files classified correctly. No duplicates. Status: Passed

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-07	Integration: Supplier Resolution Sync	Verify supplier response properly updates SME dashboard state	1. SME requests verification for "Minyak Sapi" 2. Switch to supplier view 3. Submit response: "Halal, plant-derived" 4. Return to SME view	• SME ingredient card updates to Halal • Confidence updates to 95% • Export eligibility re-evaluated	Sync successful. Status updated correctly. Status: Passed
TC-08	Integration: Manual Override Audit Trail	Verify manual declarations are properly logged with Gemini validation	1. Click "Manual Check" on ambiguous item 2. Declare status "Halal" with notes 3. Submit declaration	• Card updates to Halal (Manual) • Audit log entry created • Gemini validation passes	Update successful. Audit entry present. Status: Passed
TC-09	NFR (Schema): Export JSON Validation	Verify export bundle matches required JAKIM schema structure	1. Complete full workflow 2. Export JSON 3. Validate against mock schema	• JSON passes schema validation • All required fields present • Gemini audit trail included	Schema validation passed. All fields present. Status: Passed
TC-10	NFR (Fallback): Gemini Timeout Simulation	Verify system handles Gemini delay gracefully without errors	1. Mock Gemini delay to 3000ms 2. Trigger classification workflow	• Progress indicator shows "Processing..." • No UI freeze • Fallback to cached rules triggered	UI remained responsive. Fallback triggered correctly. Status: Passed

6. AI Output & Boundary Testing

6.1. Prompt/Response Test Pairs

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status

© UMHackathon 2026

Email: umhackathon@um.edu.my

AI-01	Classify this document: "SSM Form 24, Company: Keropok Mak Cik Sdn Bhd, Reg: 202401012345 (1234567-X)"	{ "category": "company-reg", "confidence": >=0.9, "reasoning": "Contains SSM Form 24 and valid registration number format" }	{ "category": "company-reg", "confidence": 0.94, "reasoning": "Document contains 'SSM', 'Form 24', and company registration number format matching Malaysian standards." }	Pass
AI-02	Verify ingredient: "Minyak Sapi" from supplier "XYZ Trading"	{ "status": "ambiguous", "confidence": 0.6-0.8, "reasoning": "Ambiguous term; could be beef tallow or plant oil", "action": "request_supplier_doc" }	{ "status": "ambiguous", "confidence": 0.68, "reasoning": "Ambiguous term. Could be beef tallow (requires Islamic slaughter cert) or plant-based oil. Supplier verification required.", "action": "request_supplier_doc" }	Pass
AI-03	Format for JAKIM export: { "workflowState": {...} }	Valid JSON matching JAKIM schema with: submission_id, applicant, documents[], ingredients[], compliance_summary, gemini_audit_trail[]	Generated JSON validated against mock schema; all required fields present; audit trail includes 3 steps with timestamps	Pass

6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Fields	Details
Maximum Input Size	32,000 tokens (Google Gemini 1.5 Pro context window limit)
Input used while testing	Multi-page invoice simulation: 45,000 tokens (1.4× the limit) containing repeated ingredient lists and supplier declarations

© UMHackathon 2026

Email: umhackathon@um.edu.my

Expected Behavior	• Chunk document by logical sections (header/body/footer/ingredient-list) • Process first 32K tokens prioritizing critical compliance fields • Show user warning: "Large document detected. Prioritizing ingredient list for verification." • Offer "Priority Mode" for user to select which pages to process fully
Actual Behavior	System chunked input into 3 sections, processed first 32K tokens containing company registration and ingredient list, displayed warning message in UI, allowed user to select priority pages for full processing.
Status	Pass

6.3. Adversarial/Edge Prompt Test

Test ID	Malformed/ Ambiguous Prompt	Expected Output (Acceptance Criteria)	Actual Output	Status
ADV-01	Classify: "blah blah random text no structure xyz123"	{ "category": "other", "confidence": <0.5, "reasoning": "Input does not match known document patterns" } + UI shows “Needs Manual Tagging”	Returned category: "other", confidence: 0.31. UI showed manual tagging option.	Pass
ADV-02	Verify: "xyz123unknowningredient" with no supplier context	{ "status": "ambiguous", "confidence": <0.6, "action": "manual_review", "reasoning": "Ingredient not recognized" }+ enable Manual Check button	Returned status: "ambiguous", confidence: 0.42, action: "manual_review". Manual Check button enabled.	Pass
ADV-03	User manual declaration:	Gemini validation returns { "isValid":	Validation failed with warning. Modal	Pass

© UMHackathon 2026

Email: umhackathon@um.edu.my

	status="halal", notes="" for ingredient "minyak sapi"	false, "warning": "Please specify source type for ambiguous ingredient" }	remained open with highlighted notes field.	

6.4. Hallucinating Handling

Mechanism to Detect/Mitigate Hallucinated Responses:

1. Schema Validation Layer: All Gemini outputs are validated against strict JSON schemas before rendering. Invalid structures trigger automatic retry with stricter prompt constraints.
2. Confidence Threshold Gating: Responses with confidence < 0.6 for definitive statuses (halal/haram) are automatically flagged as ambiguous and routed to manual review.
3. Cross-Reference Validation: Ingredient status is cross-checked against the mock JAKIM database. High-confidence Gemini outputs contradicting the database are flagged for escalation.
4. Reasoning Keyword Plausibility: Outputs are scanned for domain-relevant keywords (e.g., "SSM", "porcine", "JAKIM approved"). Missing keywords trigger a reasoning mismatch log.
5. Fallback Behavior: Invalid responses are logged to an immutable audit trail with reason codes. System falls back to cached rules, enables manual override, and displays clear user guidance without crashing.

© UMHackathon 2026

Email: umhackathon@um.edu.my